CSCI 112 Assignment 9 – Dynamic Memory and Memory Functions

Points: 10

Automated tests will run on GitHub Actions after every push. Check the Actions tab to see which tests pass and which fail.

The tests also run with AddressSanitizer. A memory leak, a double free or an access to freed memory will fail the whole run, even if every single assertion passes.

Instructions

1. **Fork** the repository on GitHub - `https://github.com/jpach-cs/112-ass09-template`.
2. Clone your forked repository to your local machine.
3. Open the file `code.h` and add:
    1. Include guards at the top: `#ifndef CODE_H #define CODE_H`
    2. Required includes (`<stdbool.h>`, `<stddef.h>`)
    3. The typedef `ByteBuffer` (see Part 2)
    4. All function prototypes
    5. Include guards at the bottom: `#endif`
4. Open the file `code.c`.
    1. Include `<stdlib.h>` (`malloc`, `calloc`, `realloc`, `free`), `<string.h>` (`mem*` functions, `strlen`) and `code.h`.
    2. Implement every function listed below, **in the given order** — later functions are meant to reuse the earlier ones.
        - Read the description of each function carefully.
        - Use ONLY the standard library (`stdlib.h`, `string.h`, `stdbool.h`, `stddef.h`).
        - Use the **exact growth policies and return values** specified. The tests check them.
    3. Test your implementation locally using `main.c` and compile it with the sanitizer (see "Local Testing").
5. Once all functions are complete — save your changes, then **commit** and **push** to your repository.
6. Go to the **Actions** tab on GitHub and check the test results.

Learning Objectives

After completing this lab, you will:

- Allocate memory with `malloc()` and `calloc()` and release it with `free()`
- Resize memory with `realloc()` without losing the original block when it fails

- Recognize and prevent **dangling pointers** by **nullifying** pointers after `free()`
- Avoid memory leaks, double frees and use-after-free errors
- Use `memcpy()`, `memmove()`, `memcmp()`, `memset()` and `memchr()` on raw memory blocks
- Know when `memmove()` is required instead of `memcpy()` (overlapping blocks)
- Build a growing data structure (dynamic buffer) step by step
- Work with an array of dynamically allocated strings (`char **`)

The plan: a snowball

The assignment has three parts and every part builds on the previous one.

Part	Topic	Functions	Suggested time
1	Raw allocation, `free`, dangling pointers, `realloc`	5	~25 min
2	`ByteBuffer` – a growing block of bytes	11	~55 min
3	Deep copies, searching and `char **` arrays	6	~40 min

Do **not** skip ahead. If `buffer_append()` is correct, `buffer_append_byte()` and `buffer_append_string()` are one-liners. If `buffer_reserve()` is correct, `buffer_insert()` almost writes itself.

General rules

1. **Every function that allocates must check the result.** If an allocation fails, return `NULL` / `false` and make sure you leak nothing (free what you already allocated).
2. **Never lose the only pointer to a block.** Do **not** write `p = realloc(p, size);` – if `realloc` fails you leak the old block. Use a temporary pointer.

3. Functions taking pointers must survive `NULL` arguments exactly as described below (no crash).
4. After you free something that someone else could still use through a pointer you control, **set that pointer to `NULL`**.
5. Only bytes between `0` and `size - 1` are the *content* of a `ByteBuffer`. Bytes between `size` and `capacity - 1` are *spare room* and must always be zero (`memset`).
6. Sizes are `size_t`. Every size computation has the form `count * sizeof(type)` – never hard-code `4`.

Part 1 – Raw dynamic memory

#	Function	Key concept
1	`int *alloc_ints(size_t count)`	`malloc`
2	`int *alloc_zeroed_ints(size_t count)`	`calloc`
3	`void safe_free(void **ptr)`	`free` + nullify
4	`int *resize_ints(int *arr, size_t old_count, size_t new_count)`	`realloc`, temporary pointer, `memset`
5	`int *duplicate_ints(const int *src, size_t count)`	`malloc` + `memcpy`

1. **`alloc_ints`** – Allocates room for `count` ints with `malloc()`. Returns `NULL` if `count == 0` or the allocation fails. The memory is **not** initialized.
2. **`alloc_zeroed_ints`** – Same, but every byte of the block is zero (use `calloc()`).
3. **`safe_free`** – Receives the **address of a pointer**. Frees the block the pointer points to and sets the pointer to `NULL`.
   - `safe_free(NULL)` does nothing.
   - If `*ptr` is already `NULL`, nothing happens.
   - Calling it twice on the same pointer is harmless.
   - Usage: `safe_free((void **)&p);`
4. **`resize_ints`** – Changes the block holding `old_count` ints to hold `new_count` ints.
   - `new_count == 0`: free `arr` and return `NULL`.
   - `arr == NULL`: behave like `alloc_zeroed_ints(new_count)`.

- Otherwise use `realloc()` **through a temporary pointer**. If it fails, return `NULL` and leave `arr` untouched and valid.
- Old values are kept. If the block grew, the **new** ints (`old_count` … `new_count - 1`) must be set to zero with `memset()`.

5. `duplicate_ints` – Returns a **new** block with a copy of `count` ints (`memcpy`). Returns `NULL` if `src == NULL`, `count == 0` or the allocation fails.

Part 2 – `ByteBuffer`

A `ByteBuffer` is a block of bytes that grows when you add more data than it can hold. Put this typedef in `code.h`:

```
typedef struct
{
    unsigned char *data;     /* heap block of `capacity` bytes              */
    size_t size;             /* number of bytes in use (the content)        */
    size_t capacity;         /* number of bytes allocated                   */
} ByteBuffer;
```

Invariants (always true for a valid buffer): `size <= capacity`, `capacity >= 1`, and every byte of `data` from index `size` on is `0`.

#	Function	Key concept
6	`ByteBuffer *buffer_create(size_t capacity)`	two `mallocs`, `memset`, no leak on failure
7	`bool buffer_reserve(ByteBuffer *buf, size_t min_capacity)`	growth by doubling, `realloc`, `memset`
8	`bool buffer_append(ByteBuffer *buf, const void *data, size_t n)`	`memcpy`
9	`bool buffer_append_byte(ByteBuffer *buf, unsigned char byte)`	reuse
10	`bool buffer_append_string(ByteBuffer *buf, const char *str)`	reuse, `strlen`

#	Function	Key concept
11	`int buffer_find(const ByteBuffer *buf, unsigned char byte)`	`memchr`
12	`bool buffer_equals(const ByteBuffer *a, const ByteBuffer *b)`	`memcmp`
13	`bool buffer_insert(ByteBuffer *buf, size_t pos, const void *data, size_t n)`	`memmove` + `memcpy`
14	`bool buffer_remove(ByteBuffer *buf, size_t pos, size_t n)`	`memmove` + `memset`
15	`void buffer_clear(ByteBuffer *buf)`	`memset`
16	`void buffer_destroy(ByteBuffer **buf)`	`free` x2 + nullify

6. **`buffer_create`** – Allocates the struct **and** its data block (`capacity` bytes), fills the data with zeros, sets `size = 0`. Returns `NULL` if `capacity == 0` or any allocation fails. If the struct was allocated but the data block was not, **free the struct** before returning `NULL`.
7. **`buffer_reserve`** – Makes sure the buffer can hold at least `min_capacity` bytes.
   - `buf == NULL` → `false`.
   - If `min_capacity <= capacity` → `true`, nothing changes.
   - Otherwise **double** the capacity until it is `>= min_capacity` (e.g. `4` → `8` → `16` → `32`), `realloc` the data, and zero the new bytes. Return `true`. If `realloc` fails return `false` and keep the buffer unchanged.
8. **`buffer_append`** – Appends `n` bytes from `data` at the end of the content, growing the buffer with `buffer_reserve(buf, size + n)` if needed.
   - `buf == NULL`, or `data == NULL` while `n > 0` → `false`.
   - `n == 0` → `true`, nothing changes.
9. **`buffer_append_byte`** – Appends one byte (use `buffer_append`).
10. **`buffer_append_string`** – Appends the characters of `str` **without** the terminating `'\0'` (use `buffer_append` and `strlen`). `str == NULL` → `false`. An empty string → `true`, nothing changes.
11. **`buffer_find`** – Index of the **first** occurrence of `byte` in the **content** (never look at the spare room!). Returns `-1` if not found or `buf == NULL`.
12. **`buffer_equals`** – `true` if both buffers have the same `size` and the same

content bytes. The capacities do not matter. Two empty buffers are equal. If any argument is `NULL` → `false`.

13. `buffer_insert` – Inserts `n` bytes from `data` so that they start at index `pos`; the old bytes from `pos` on move to the right. Valid positions are `0` … `size` (`pos == size` means "append").
    - `buf == NULL`, `pos > size`, or `data == NULL` while `n > 0` → `false` (buffer unchanged).
    - `n == 0` → `true`, nothing changes.
    - Grow with `buffer_reserve` if needed. The tail **overlaps** its own destination, so use `memmove()`.
14. `buffer_remove` – Removes `n` bytes starting at index `pos`; the following bytes move to the left.
    - `buf == NULL`, `pos > size` or `pos + n > size` → `false` (buffer unchanged). Write the check so that it cannot overflow (`n > size - pos`).
    - `n == 0` → `true`, nothing changes.
    - After the move the `n` bytes that became unused at the end must be set to zero.
15. `buffer_clear` – Sets `size` to `0` and zeroes the **whole** data block. The capacity stays. `NULL` → do nothing.
16. `buffer_destroy` – Receives the **address of your pointer**. Frees the data block, then the struct, then sets the caller's pointer to `NULL`. `NULL` argument, or `*buf == NULL` → do nothing. Calling it twice must be harmless.

Part 3 – Copies, searching and arrays of strings

#	Function	Key concept
17	`ByteBuffer *buffer_clone(const ByteBuffer *src)`	deep copy
18	`ByteBuffer *buffer_concat(const ByteBuffer *a, const ByteBuffer *b)`	reuse, exact capacity
19	`size_t buffer_count(const ByteBuffer *buf, unsigned char byte)`	`memchr` in a loop
20	`bool buffer_shrink_to_fit(ByteBuffer *buf)`	`realloc` to a smaller block

#	Function	Key concept
21	`char **buffer_split(const ByteBuffer *buf, unsigned char delim, size_t *out_count)`	array of heap strings
22	`void free_tokens(char ***tokens, size_t count)`	freeing a 2-level structure + nullify

17. `buffer_clone` – A **deep copy**: a new struct with its **own** data block, same `capacity`, same content (`size` bytes copied with `memcpy`). `src == NULL` → `NULL`. Changing the clone must never change the source (and vice versa).
18. `buffer_concat` – A new buffer holding the content of `a` followed by the content of `b`. Its capacity is **exactly** `a->size + b->size` (but at least `1`). Neither source is modified. Any `NULL` argument → `NULL`.
19. `buffer_count` – How many times `byte` occurs in the **content**. Use `memchr()` repeatedly, continuing after the last hit. `NULL` → `0`.
20. `buffer_shrink_to_fit` – Reduces the capacity to `size` (but at least `1`) using `realloc` through a temporary pointer. The content stays. `NULL` → `false`; success (also when nothing had to change) → `true`.
21. `buffer_split` – Splits the content at every `delim` byte and returns a **heap-allocated array of heap-allocated strings**.
    - A buffer with k delimiters gives $k + 1$ tokens. Empty tokens are kept: `"a,bb,,ccc"` split at `','` gives `"a"`, `"bb"`, `""`, `"ccc"`. `"a,"` gives `"a"` and `""`. An **empty buffer** gives exactly **one** empty token.
    - Every token is a separate `malloc`ed, `'\0'`-terminated copy (`memcpy` + terminator). Tokens never point into the buffer; the buffer is not modified.
    - The array of pointers itself must grow while you discover tokens — start small (for example 4 pointers) and use `realloc` (doubling).
    - `*out_count` receives the number of tokens. On any failure (including `buf == NULL` or `out_count == NULL`) return `NULL`; if `out_count != NULL` set `*out_count = 0`, and **free everything you allocated so far**.
22. `free_tokens` – Frees every token, then the array itself, then sets the caller's pointer to `NULL`. `tokens == NULL` or `*tokens == NULL` → do nothing. Calling it twice must be harmless.

Background

Allocation functions (`<stdlib.h>`)

Function	What it does
`void *malloc(size_t bytes)`	Allocates `bytes` bytes, contents **uninitialized**. Returns `NULL` on failure.
`void *calloc(size_t count, size_t size)`	Allocates `count * size` bytes, **all zero**. Returns `NULL` on failure.
`void *realloc(void *ptr, size_t bytes)`	Changes the size of a block, keeping the old contents. May **move** the block. Returns `NULL` on failure (the old block is then still valid!).
`void free(void *ptr)`	Releases a block. `free(NULL)` is allowed and does nothing.

```c
int *p = malloc(10 * sizeof(int));      /* sizeof(int), not 40 */
if (p == NULL)
{
    return NULL;                        /* always check */
}
```

The `realloc` pattern

```c
/* WRONG: if realloc fails, p becomes NULL and the old block is leaked */
p = realloc(p, new_bytes);

/* RIGHT: keep the old pointer until you know it worked */
int *tmp = realloc(p, new_bytes);
if (tmp == NULL)
{
    return false;       /* p is still valid */
}
p = tmp;
```

After a successful `realloc`, the **old** pointer must not be used any more – the block may have moved.

Dangling pointers and nullifying

A **dangling pointer** points to memory that has already been freed. Using it is undefined behavior (use-after-free), and freeing it again is a **double free**.

```c
int *p = malloc(sizeof(int));
free(p);
```

```
*p = 5;          /* use after free - undefined behavior              */
free(p);         /* double free - crash                              */
```

Freeing does **not** change the pointer variable. The cure is to **nullify** it, because `free(NULL)` is harmless and `if (p != NULL)` can then tell whether the pointer is usable:

```
free(p);
p = NULL;
```

To do that inside a helper function you must pass the **address of the pointer** (a pointer to a pointer), otherwise the helper would only change its own copy:

```
void safe_free(void **ptr)
{
    free(*ptr);
    *ptr = NULL;        /* changes the caller's variable */
}

safe_free((void **)&p);
```

Note that nullifying only protects the pointer you nullify. Other copies (aliases) of the same address are still dangling.

Memory functions (`<string.h>`)

Used for working with raw memory blocks (not only strings – these functions do not stop at `'\0'`).

Function	Description	Example
`memcpy(dest, src, n)`	Copies `n` bytes. The blocks **must not overlap**.	`memcpy(buf2, buf1, n);`
`memmove(dest, src, n)`	Safe copy – works even when the blocks overlap.	`memmove(a, b, 5);`
`memcmp(a, b, n)`	Compares `n` bytes. Returns `0` when they are equal.	`if (memcmp(a, b, 4) == 0)`
`memset(ptr, val, n)`	Fills `n` bytes with the byte value `val`.	`memset(buf, 0, 10);`
`memchr(ptr, val, n)`	Finds the first byte equal to `val` in the first `n` bytes. Returns a pointer to it, or `NULL`.	`p = memchr(buf, 'x', 20);`

```
/* index of a byte found with memchr: subtract the start of the block */
const unsigned char *found = memchr(buf->data, 'x', buf->size);
if (found != NULL)
{
```

```
    size_t index = (size_t)(found - buf->data);
}
```

Why `memmove` for insert and remove? Shifting the tail of a buffer by a few bytes copies a block onto **itself**:

```
before:  a b c d e f . .          insert "XY" at 2
after:   a b X Y c d e f          tail "cdef" moves 2 bytes to the right
```

The source (`c d e f`) and the destination (`c d e f` shifted) overlap, so `memcpy` is not allowed here.

Growth by doubling

Growing a buffer by one byte at a time would call `realloc` very often. Doubling the capacity makes appends cheap on average:

```
capacity 4, need 5   ->  8
capacity 4, need 17  ->  4 -> 8 -> 16 -> 32
capacity 1, need 100 ->  1 -> 2 -> 4 -> 8 -> 16 -> 32 -> 64 -> 128
```

Allocation of a 2-level structure

`buffer_split` returns `char **`. Remember that there are `count + 1` separate allocations to free:

```
tokens    [ ptr0 ][ ptr1 ][ ptr2 ]       (1 block: the array of pointers)


            "abc"   "de"    ""            (count blocks: one per string)
```

Free the strings first, then the array – otherwise you lose the pointers to the strings.

Only for testing

Local Testing (Sandbox)

Compile with the sanitizer – it tells you about leaks, double frees and use-after-free:

```
gcc -g -Wall -Wextra -fsanitize=address,undefined main.c code.c -o main
./main
```

Create test cases in main.c:

```
#include <stdio.h>
#include "code.h"
```

```c
int main()
{
    // Part 1
    int *a = alloc_zeroed_ints(3);
    a = resize_ints(a, 3, 6);
    a[5] = 42;
    printf("a[0]=%d a[5]=%d\n", a[0], a[5]);
    safe_free((void **)&a);
    printf("a after safe_free: %p\n", (void *)a);

    // Part 2
    ByteBuffer *b = buffer_create(2);
    buffer_append_string(b, "Hello");
    buffer_append_byte(b, ' ');
    buffer_append_string(b, "World");
    buffer_insert(b, 5, ",", 1);
    printf("size=%zu capacity=%zu find('W')=%d\n", b->size, b->capacity, buffer_find(b, 'W')
    printf("content: %.*s\n", (int)b->size, (char *)b->data);

    // Part 3
    buffer_clear(b);
    buffer_append_string(b, "one,two,,four");
    size_t n = 0;
    char **tokens = buffer_split(b, ',', &n);
    for (size_t i = 0; i < n; i++)
    {
        printf("token %zu: \"%s\"\n", i, tokens[i]);
    }
    free_tokens(&tokens, n);

    ByteBuffer *copy = buffer_clone(b);
    printf("clone equals original: %d\n", buffer_equals(b, copy));

    buffer_destroy(&copy);
    buffer_destroy(&b);
    return 0;
}
```

Hints: - `printf("%.*s", (int)len, ptr)` prints `len` characters – a `ByteBuffer` is **not** `'\0'`-terminated. - If the sanitizer says "detected memory leaks", it also prints where the leaked block was allocated – start there. - Write each function, compile, and test it **before** moving to the next one.